

Stay Away from SQL Injections



SQL injection is a common hacking technique that places malicious code in SQL statements, via Web page input. It can destroy your database. Three SQL injection vulnerabilities and their remedies are discussed in this article.

SQL injection is a code injection technique that exploits the security vulnerability of the connected database layer of an application. This vulnerability allows poorly filtered user inputs with undesirable escape characters embedded within SQL statements to get executed by the database server. User inputs with an inappropriate data type also create an SQL injection vulnerability problem. Unfiltered user input through a Web page is a common source of this type of security breach, as it makes the database accessible to hackers to gain control over it. This security loophole generally occurs when one programming or scripting language is embedded inside another. SQL injection vulnerability is also known as SQL insertion attack.

SQL injection typically occurs when a user-supplied input, like the name, is purposefully modified to a specific MySQL statement to get executed within the Web database. For example, a user form for user name may be designed to enter the user's name for credential verification. But instead of a simple alphanumeric literal string, a user may provide a string to delete a table, as shown in Figure 1.

With this form input entry, the server will drop the entire user's table. This is an example of how commanding and hazardous an SQL injection can be and shows the necessity of regular database backups.

Here I shall discuss only three of the SQL injection features and their possible remedies. These are: bypassing query logic, retrieving protected data and multiple query

injections. All these vulnerabilities are internal to SQL and intrinsic to embedded programming structures but arise mainly due to careless coding. Proper systematic goal-oriented design methodology can reduce the chance of these to a great extent and make the database secure from external attacks.

Retrieving hidden data

SQL injection can be demonstrated directly on SQL database prompt. For example, if we consider an SQL query to browse a user table for a user name, and assign a user name variable string as:

"HelloWorld" or 1=1-', the SQL SELECT statement:

```
set @id1 := "HelloWorld", @id2 := "1=1--" ;
select * from users where username= @id1 or @id2;
```

...it will display all records from the user's table irrespective of the value of the user name field. Even if the user name 'HelloWorld' is absent in the user table, all the records will still be displayed.

Retrieving hidden data: Web page mode

The above vulnerability can be explored through a Web-based login module to bypass a user's name pattern matching restriction. For example, if there is a login form *login.html* as shown below:

```
login.html
<form action='login.php' method="post">
<input type="name" name="username" required="required"/>
<input type="submit" value="Submit"/>
</form>
```



Figure 1: SQL injection to drop a table